

전자전기컴퓨터설계실험II

실험 전 보고서

LAB5: STM32_1 및 FPGA 순차회로

학과	전자전기컴퓨터공학부
이름/학번	김영서 / 2025440031
작성일	2026년 10월 19일
대상	STM32 인터럽트·PWM, FPGA 피아노·신호등·ADC

검증 환경

FPGA 시뮬레이터: Icarus Verilog

Vivado 구현·XDC: 본 실험에서 진행 예정

1. 실험 목적 및 공통 검증 조건

1.1 실험 목적

이번 실험의 첫 번째 목적은 STM32에서 폴링 방식으로 구현했던 버튼 입력과 LED 시간 제어를 인터럽트 방식으로 다시 구성하는 것이다. 사용자 버튼이 눌렸을 때는 외부 인터럽트 서비스 루틴이 실행되고, TIM6은 100 ms마다 주기적으로 인터럽트를 발생시킨다. 두 인터럽트가 공유하는 상태값을 이용해 내장 LED의 상태 변경 간격을 1초, 2초, 4초, 8초 순서로 바꾼다. 이를 통해 메인 반복문이 계속 입력과 시간을 확인하지 않아도 주변장치의 사건에 맞추어 코드가 실행되는 구조를 이해한다. 인터럽트의 우선순위와 공유 변수의 의미는 Cortex-M 계열의 예외 처리 구조와 관련된다[1].

두 번째 목적은 TIM3 채널 1의 PWM 출력을 사용하여 PB4에서 1 kHz 파형을 만들고, 버튼을 누를 때마다 듀티비를 25%, 50%, 75% 순서로 변경하는 것이다. C 디버거의 중단점, 한 줄 실행, 변수 및 조사식 기능을 이용해 버튼 인터럽트 진입과 PWM 비교값 변화를 확인한다.
세 번째 목적은 FPGA에서 피아노, 신호등 제어기, 외부 ADC 직렬 인터페이스를 Verilog로 모듈화하고 테스트벤치로 검증하는 것이다. 피아노 회로에서는 선택한 음계에 맞는 사각파를 생성하고, 신호등 회로에서는 1초 기준 신호와 상태 순서를 결합하며, ADC 회로에서는 채널 선택 명령을 16비트 직렬 전송하고 수신한 12비트 결과를 저장한다. 카운터, 조합 논리, 유한상태 순서 및 계층형 모듈 구성을 통해 순차회로 설계 원리를 확인한다[2].

1.2 실험 대상과 공통 조건

구분	공통 조건
STM32 보드	NUCLEO-G474RE
STM32 입력	사용자 버튼 PC13, HIGH 활성
STM32 LED	내장 LED PA5, HIGH일 때 켜짐
주기 인터럽트	TIM6, 100 ms마다 호출, 우선순위 4
버튼 인터럽트	button_isr_p7(), 우선순위 7, 200 ms 디바운스
PWM	TIM3 채널 1, PB4, 1 kHz, 한 주기 1000카운트
FPGA 기준 클록	clk_50mhz, 50 MHz
FPGA 리셋	rst_p, HIGH 활성
HDL 검증	테스트벤치의 입력은 하강 에지에서 변경하고 상승 에지 후 결과를 판정
구현 범위	현재 예비 단계에서는 소스와 테스트벤치 및 디지털 시뮬레이션까지 수행

표 1. 공통 실험 조건과 사전 검증 범위

STM32_1 과제3과 과제4는 같은 버튼 인터럽트 함수를 서로 다른 목적으로 사용하므로 각각 별도의 프로그램으로 빌드하고 확인한다. 과제3을 실행할 때는 과제4의 PWM 변경 코드를 제거하고, 과제4를 실행할 때는 과제3의 LED 간격 변경 코드를 제거한다.

FPGA 테스트벤치에서는 실행 시간을 줄이기 위해 클록 관련 매개변수를 축소한다. 따라서 시뮬레이션은 논리

순서와 클록 수가 맞는지를 검증하며, 실제 1초·1 kHz·음계 주파수와 외부 장치의 아날로그 특성을 직접 측정한 결과는 아니다. Vivado 프로젝트 생성, 핀 배치, 합성, 구현, 타이밍 분석, 비트스트림 생성과 보드 시연은 다음 단계에서 수행한다. 교재와 장비의 검증된 핀맵을 아직 대조하지 않았으므로 이번 예비보고서에는 임의의 XDC를 작성하지 않는다.

1.3 공통 판정 기준

1. 설계 소스가 문법 오류 없이 컴파일되어야 한다.
2. 자체 검사 테스트벤치가 모든 지정 입력을 검사한 뒤 PASS를 출력해야 한다.
3. 파형에서 리셋, 정상 동작, 경계값 및 예외 입력을 확인해야 한다.
4. 시뮬레이션 결과와 실제 보드 결과를 구분해 기록해야 한다.
5. 본 실험에서는 Vivado의 Synthesis, Implementation, DRC, Timing, Bitstream 결과와 보드 시연을 추가로 확보해야 한다.

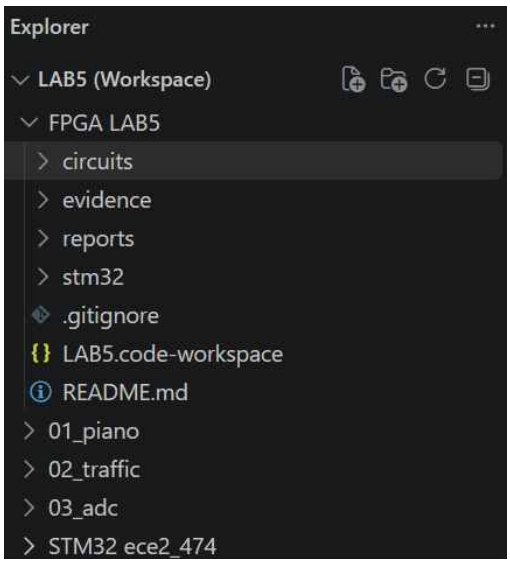


그림 1. 사용자가 직접 캡처한 LAB5 VS Code 작업공간

2. STM32_1 과제3 · 버튼·TIM6 인터럽트로 LED 간격 변경

2.1 과제 요구사항

STM32_0 과제1의 기능을 폴링 없이 인터럽트로 재현한다. PC13 버튼을 누를 때마다 PA5 내장 LED의 상태 변경 간격이 1초 → 2초 → 4초 → 8초 → 1초로 순환해야 한다. 여기서 1초는 LED가 켜진 뒤 꺼질 때까지 또는 꺼진 뒤 켜질 때까지의 시간이며, 완전한 켜짐·꺼짐 한 주기는 그 두 배다. 버튼을 계속 누르고 있어도 한 번만 변경되어야 한다.

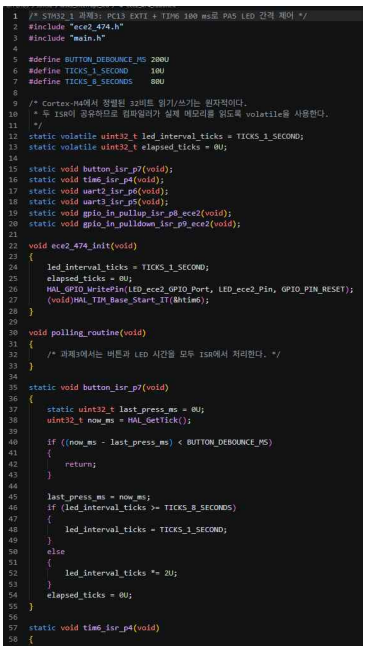
2.2 프로그램 구조

ece2_474_init()에서 HAL_TIM_Base_Start_IT(&htim6);을 호출해 TIM6 인터럽트를 시작한다.
polling_routine()에서는 버튼을 읽거나 LED 시간을 계산하지 않는다. 버튼 사건과 LED 시간 계산은 다음 두 ISR에서 처리한다.

• button_isr_p7()은 마지막으로 인정한 버튼 입력과 현재 시간의 차이가 200 ms 이상일 때만 입력을 인정한다. 인정된 입력마다 간격 인덱스를 다음 값으로 이동하고, 새 간격을 버튼 입력 시점부터 명확히 적용하기 위해 TIM6 누적 횟수를 0으로 초기화한다.

• tim6_isr_p4()는 100 ms마다 누적 횟수를 1 증가시킨다. 누적 횟수가 선택된 기준값에 도달하면 횟수를 0으로 되돌리고 PA5를 토글한다.

• 두 ISR이 함께 사용하는 led_interval_ticks와 elapsed_ticks는 함수 밖에 volatile uint32_t로 선언한다. volatile은 매번 메모리의 값을 읽게 하지만, 여러 연산 전체를 원자적으로 만드는 기능은 아니다.
TIM6의 우선순위는 4이고 버튼의 우선순위는 7이다. Cortex-M의 숫자가 작은 우선순위가 더 높으므로 두 인터럽트가 겹치면 TIM6이 먼저 처리될 수 있다. 공유값은 32비트 정렬 변수로 단순하게 읽고 쓰고, ISR 안에서는 printf, HAL_Delay, 입력 대기와 같은 긴 처리를 사용하지 않는다.



```c
/* STM32_1 과제3: PC13 EXTI + TIM6 100 ms로 PA5 LED 간격 제어 */
#include "ece2_474.h"
#include "main.h"

#define BUTTON_DEBOUNCE_MS 200U
#define TICKS_1_SECOND     10U
#define TICKS_8_SECONDS    80U

/* Cortex-M4에서 정렬된 32비트 읽기/쓰기는 원자적이다.
 * 두 ISR이 공유하므로 컴파일러가 실제 메모리를 읽도록 volatile을 사용한다.
 */
static volatile uint32_t led_interval_ticks = TICKS_1_SECOND;
static volatile uint32_t elapsed_ticks = 0U;

static void button_isr_p7(void);
static void tim6_isr_p4(void);
static void uart2_isr_p6(void);
static void uart3_isr_p5(void);
static void gpio_in_pullup_isr_p8_ece2(void);
static void gpio_in_pulldown_isr_p9_ece2(void);

void ece2_474_init(void)
{
    led_interval_ticks = TICKS_1_SECOND;
    elapsed_ticks = 0U;
    HAL_GPIO_WritePin(LED_ece2_GPIO_Port, LED_ece2_Pin, GPIO_PIN_RESET);
    (void)HAL_TIM_Base_Start_IT(&htim6);
}

void polling_routine(void)
{
    /* 과제3에서는 버튼과 LED 시간을 모두 ISR에서 처리한다. */
}

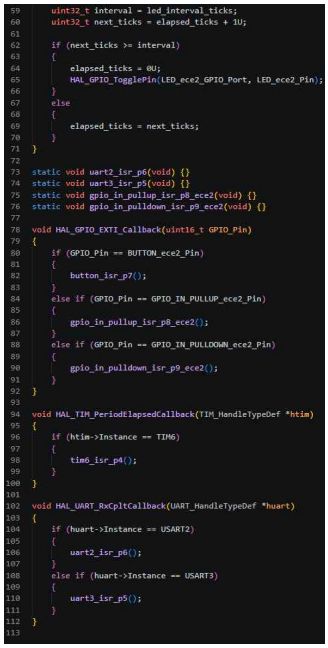
static void button_isr_p7(void)
{
    static uint32_t last_press_ms = 0U;
    uint32_t now_ms = HAL_GetTick();

    if ((now_ms - last_press_ms) < BUTTON_DEBOUNCE_MS)
    {
        return;
    }

    last_press_ms = now_ms;
    if (led_interval_ticks >= TICKS_8_SECONDS)
    {
        led_interval_ticks = TICKS_1_SECOND;
    }
    else
    {
        led_interval_ticks *= 2U;
    }
    elapsed_ticks = 0U;
}

static void tim6_isr_p4(void)
{
```

그림 2(a-1). 사용자가 직접 캡처한 STM32_1 과제3 코드 전반부



```
    uint32_t interval = led_interval_ticks;
    uint32_t next_ticks = elapsed_ticks + 1U;

    if (next_ticks >= interval)
    {
        elapsed_ticks = 0U;
        HAL_GPIO_TogglePin(LED_ece2_GPIO_Port, LED_ece2_Pin);
    }
    else
    {
        elapsed_ticks = next_ticks;
    }
}

static void uart2_isr_p6(void) {}
static void uart3_isr_p5(void) {}
static void gpio_in_pullup_isr_p8_ece2(void) {}
static void gpio_in_pulldown_isr_p9_ece2(void) {}

void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
    if (GPIO_Pin == BUTTON_ece2_Pin)
    {
        button_isr_p7();
    }
    else if (GPIO_Pin == GPIO_IN_PULLUP_ece2_Pin)
    {
        gpio_in_pullup_isr_p8_ece2();
    }
    else if (GPIO_Pin == GPIO_IN_PULLDOWN_ece2_Pin)
    {
        gpio_in_pulldown_isr_p9_ece2();
    }
}

void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
    if (htim->Instance == TIM6)
    {
        tim6_isr_p4();
    }
}
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
{
    if (huart->Instance == USART2)
    {
        uart2_isr_p6();
    }
    else if (huart->Instance == USART3)
    {
        uart3_isr_p5();
    }
}
```

그림 2(a-2). 사용자가 직접 캡처한 STM32_1 과제3 코드 후반부

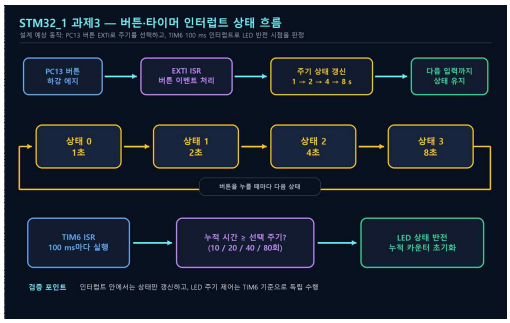


그림 2(b). PC13 버튼과 TIM6 인터럽트에 따른 1·2·4·8초 상태 흐름



그림 2(c). 과제3·과제4의 실제 ARM GCC Debug 빌드 성공 결과

2.3 예상 동작

선택 단계	led_interval_ticks	TIM6 호출 주기	LED 상태 변경 간격	LED 전체 점멸 주기	다음 버튼 입력 후
0	10	100 ms	1 s	2 s	2 s 선택
1	20	100 ms	2 s	4 s	4 s 선택
2	40	100 ms	4 s	8 s	8 s 선택
3	80	100 ms	8 s	16 s	1 s 선택

표 2. STM32_1 과제3 LED 상태 변경 간격 예상값

초기값은 1초로 설정한다. 예를 들어 초기화 뒤 TIM6 ISR이 10회 호출되면 LED가 한 번 토글되고, 버튼을 한 번 누르면 누적값이 0으로 초기화된 뒤 20회의 호출 후 다음 토글이 발생한다. 버튼 접점이 짧은 시간 동안 여러 번 변해도 200 ms 이내의 재호출은 무시한다.

2.4 사전 검증 및 본 실험 확인 방법

예비 단계에서는 프로젝트가 정상적으로 빌드되는지, ISR 안의 간격 순환식과 카운터 조건이 요구사항과 일치하는지 정적 검토한다. 보드 실행 결과는 아직 작성하지 않는다. 본 실험에서는 다음 장면을 확보한다.

1. 빌드가 성공한 터미널 또는 Build 결과

2. button_isr_p7()에 둔 중단점에서 실행이 멈춘 화면
3. Variables 또는 Watch에서 간격 인덱스, led_interval_ticks, elapsed_ticks가 바뀌는 화면
4. 버튼을 누를 때마다 PA5 토글 간격이 1·2·4·8초로 순환하는 영상
예상과 실제 값의 오차는 스마트폰 영상의 시간 표시 또는 계측 결과로 비교한다. 디버거 중단점에서 멈춘 시간은 정상 동작의 시간 측정값에 포함하지 않는다.

3. STM32_1 과제4 · 버튼으로 PWM 듀티비 변경

3.1 과제 요구사항

TIM3 채널 1을 시작하여 PB4에서 1 kHz PWM을 출력한다. 초기 듀티비는 25%로 하고, PC13 버튼을 누를 때마다 25% → 50% → 75% → 25% 순서로 변경한다. 과제3의 버튼 처리와 동시에 실행하지 않고 과제4 전용 코드로 확인한다.

3.2 프로그램 구조

ece2_474_init()에서 다음 두 동작을 수행한다.
1. __HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_1, 250U);
2. HAL_TIM_PWM_Start(&htim3, TIM_CHANNEL_1);
버튼 ISR에서는 200 ms 디바운스를 적용한 뒤 듀티 인덱스를 다음 상태로 이동하고 __HAL_TIM_SET_COMPARE()에 250, 500, 750 중 하나를 전달한다. PWM 한 주기가 1000카운트이므로 비교값을 주기 카운트로 나눈 값이 듀티비가 된다. 초기값이 25%이므로 첫 번째 인정된 버튼 입력 후 값은 50%가 되어야 한다.



```c
/* STM32_1 과제4: PC13 EXTI로 PB4 TIM3 CH1 PWM 듀티비 순환 */
#include "ece2_474.h"
#include "main.h"

#define BUTTON_DEBOUNCE_MS 200U
#define PWM_COMPARE_25     250U
#define PWM_COMPARE_50     500U
#define PWM_COMPARE_75     750U

/* 디버거의 Variables/Watch에서 pwm_compare를 확인한다. */
static volatile uint32_t pwm_compare = PWM_COMPARE_25;

static void button_isr_p7(void);
static void tim6_isr_p4(void);
static void uart2_isr_p6(void);
static void uart3_isr_p5(void);
static void gpio_in_pullup_isr_p8_ece2(void);
static void gpio_in_pulldown_isr_p9_ece2(void);

void ece2_474_init(void)
{
    pwm_compare = PWM_COMPARE_25;
    __HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_1, pwm_compare);
    (void)HAL_TIM_PWM_Start(&htim3, TIM_CHANNEL_1);
}

void polling_routine(void)
{
    /* 과제4에서는 버튼 인터럽트가 PWM 비교값을 직접 변경한다. */
}

static void button_isr_p7(void)
{
    static uint32_t last_press_ms = 0U;
    uint32_t now_ms = HAL_GetTick();

    if ((now_ms - last_press_ms) < BUTTON_DEBOUNCE_MS)
    {
        return;
    }
    last_press_ms = now_ms;

    if (pwm_compare == PWM_COMPARE_25)
    {
        pwm_compare = PWM_COMPARE_50;
    }
    else if (pwm_compare == PWM_COMPARE_50)
    {
        pwm_compare = PWM_COMPARE_75;
    }
    else
    {
        pwm_compare = PWM_COMPARE_25;
    }

    __HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_1, pwm_compare);
}
```

그림 3(a-1). 사용자가 직접 캡처한 STM32_1 과제4 코드 전반부

```
    __HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_1, pwm_compare);
}

static void tim6_isr_p4(void)
{
    /* 과제4에서는 TIM6를 사용하지 않는다. */
}

static void uart2_isr_p6(void) {}
static void uart3_isr_p5(void) {}
static void gpio_in_pullup_isr_p8_ece2(void) {}
static void gpio_in_pulldown_isr_p9_ece2(void) {}

void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
    if (GPIO_Pin == BUTTON_ece2_Pin)
    {
        button_isr_p7();
    }
    else if (GPIO_Pin == GPIO_IN_PULLUP_ece2_Pin)
    {
        gpio_in_pullup_isr_p8_ece2();
    }
    else if (GPIO_Pin == GPIO_IN_PULLDOWN_ece2_Pin)
    {
        gpio_in_pulldown_isr_p9_ece2();
    }
}

void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
    if (htim->Instance == TIM6)
    {
        tim6_isr_p4();
    }
}

void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
{
    if (huart->Instance == USART2)
    {
        uart2_isr_p6();
    }
    else if (huart->Instance == USART3)
    {
        uart3_isr_p5();
    }
}
```

그림 3(a-2). 사용자가 직접 캡처한 STM32_1 과제4 코드 후반부

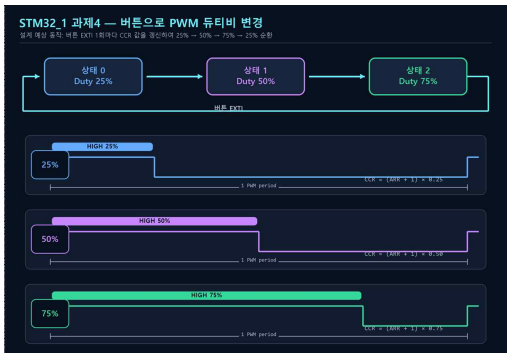


그림 3(b). 1 kHz PWM의 25%·50%·75% 예상 비교 파형

3.3 예상값

상태	TIM3 비교값	듀티비	1 kHz에서 HIGH 시간	LOW 시간	다음 버튼 입력 후
0	250	25%	0.25 ms	0.75 ms	50%
1	500	50%	0.50 ms	0.50 ms	75%
2	750	75%	0.75 ms	0.25 ms	25%

표 3. STM32_1 과제4 PWM 듀티비 예상값

3.4 C 디버거 및 계측 계획

1. button_isr_p7()의 듀티 인덱스 변경 줄에 F9로 중단점을 설정한다.
2. F5로 디버깅을 시작하고 main에서 멈추면 F5를 한 번 더 누른다.
3. 보드 버튼을 눌러 버튼 ISR의 중단점에서 멈추는지 확인한다.
4. Variables 또는 Watch에서 듀티 인덱스와 비교값을 확인한다.
5. F10으로 한 줄씩 실행해 250 → 500 → 750 → 250 변화가 발생하는지 확인한다. 함수 안으로 들어갈 때는 F11, 계속 실행은 F5를 사용한다.
6. 중단점을 해제하고 정상 실행 상태로 만든 뒤 PB4와 GND를 계측기에 연결한다.
7. 버튼을 누를 때마다 주파수가 약 1 kHz로 유지되고 HIGH 시간이 0.25 ms, 0.50 ms, 0.75 ms로 바뀌는지 측정한다.
8. Shift+F5로 디버깅을 종료한다.

4. FPGA · 피아노

4.1 회로 구조

피아노 회로는 note_select, square_tone, piano의 세 모듈로 구성한다.

• note_select는 8비트 one-hot key 입력을 음계 주파수로 변환하는 조합회로다.

• square_tone은 50 MHz 클록을 CLK_HZ/(2×frequency_hz)만큼 세고 출력을 반전해 약 50% 듀티의 사각파를 만든다.

• piano는 두 하위 모듈을 연결하고 piezo 출력을 제공한다.

키 입력이 0이거나 두 개 이상 동시에 눌린 경우에는 주파수를 0으로 정해 무음으로 처리한다. 음계가 바뀌면 카운터와 출력을 0으로 초기화하고 새 음의 반주기를 처음부터 센다.

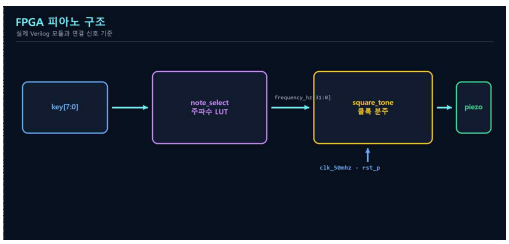


그림 6. 피아노 회로의 계층 구조와 신호 흐름

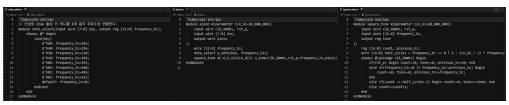
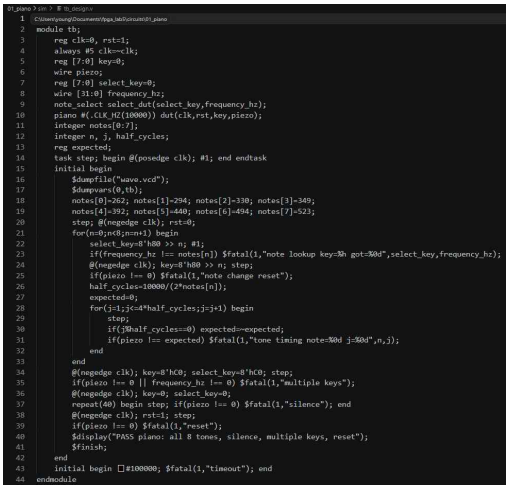


그림 7(a). 사용자가 직접 캡처한 피아노 Verilog 설계 코드



```
module tb;
    reg clk=0, rst=1;
    always #5 clk=~clk;
    reg [7:0] key=0;
    wire piezo;
    reg [7:0] select_key=0;
    wire [31:0] frequency_hz;
    note_select select_dut(select_key,frequency_hz);
    piano #(.CLK_HZ(100000)) dut(clk,rst,key,piezo);
    integer notes[0:7];
    integer n, j, half_cycles;
    reg expected;
    task step; begin @(posedge clk); #1; end endtask
    initial begin
        $dumpfile("wave.vcd");
        $dumpvars(0,tb);
        notes[0]=262; notes[1]=294; notes[2]=330; notes[3]=349;
        notes[4]=392; notes[5]=440; notes[6]=494; notes[7]=523;
        step; @(negedge clk); rst=0;
        for(n=0;n<8;n=n+1) begin
            select_key=8'h80 >> n; #1;
            if(frequency_hz !== notes[n]) $fatal(1,"note lookup key=%h got=%0d",select_key,frequency_hz);
            @(negedge clk); key=8'h80 >> n; step;
            if(piezo !== 0) $fatal(1,"note change reset");
            half_cycles=10000/(2*notes[n]);
            expected=0;
            for(j=1;j<=4*half_cycles;j=j+1) begin
                step;
                if(j%half_cycles==0) expected=~expected;
                if(piezo !== expected) $fatal(1,"tone timing note=%0d j=%0d",n,j);
            end
        end
        @(negedge clk); key=8'hC0; select_key=8'hC0; step;
        if(piezo !== 0 || frequency_hz !== 0) $fatal(1,"multiple keys");
        @(negedge clk); key=0; select_key=0;
        repeat(40) begin step; if(piezo !== 0) $fatal(1,"silence"); end
        @(negedge clk); rst=1; step;
        if(piezo !== 0) $fatal(1,"reset");
        $display("PASS piano: all 8 tones, silence, multiple keys, reset");
        $finish;
    end
    initial begin #100000; $fatal(1,"timeout"); end
endmodule
```

그림 7(b). 사용자가 직접 캡처한 피아노 self-checking 테스트벤치

4.2 음계별 예상값

key	음계	목표 주파수	50 MHz에서 half_cycles
8'h80	도(C4)	262 Hz	95,419
8'h40	레(D4)	294 Hz	85,034
8'h20	미(E4)	330 Hz	75,757
8'h10	파(F4)	349 Hz	71,633
8'h08	솔(G4)	392 Hz	63,775
8'h04	라(A4)	440 Hz	56,818
8'h02	시(B4)	494 Hz	50,607
8'h01	높은 도(C5)	523 Hz	47,801
그 외	무음	0 Hz	출력 LOW 유지

표 4. 피아노 음계별 주파수와 분주값

정수 나눗셈 때문에 실제 출력 주파수에는 매우 작은 양자화 오차가 생길 수 있다. 보드에서는 음이 구분되는지와 측정 주파수가 목표값에 가까운지를 확인한다.

4.3 테스트벤치 검증

시뮬레이션에서는 CLK_HZ를 10,000으로 축소해 실행 시간을 줄인다. 8개 one-hot 키의 주파수 선택값을 모두 검사하고, 각 음에서 계산된 반주기마다 piezo가 반전되는지 네 반주기 동안 확인한다. 8'hC0처럼 여러 키가 동시에 입력된 경우, 키가 없는 경우, 리셋을 가한 경우에는 출력이 LOW인지 검사한다. 예상 종료 메시지는 다음과 같다.

```
PASS piano: all 8 tones, silence, multiple keys, reset
```



그림 8. 사용자가 직접 실행한 피아노 self-checking 시뮬레이션 PASS 결과



그림 9. 사용자가 직접 확인한 피아노 key·frequency_hz·piezo 파형

본 실험에서는 Vivado 시뮬레이션으로 같은 순서를 확인한 뒤, 검증된 장비 핀맵에 따라 버튼과 피에조를 연결하고 8개 음이 서로 다르게 출력되는 영상을 촬영한다. 현재 단계에서는 핀 정보가 확인되지 않았으므로 XDC와 보드 결과를 제시하지 않는다.

5. FPGA · 신호등 제어

5.1 회로 구조

신호등 회로는 second_tick, traffic_sequence, traffic_lights, traffic으로 구성한다.

• second_tick은 50 MHz 클록 50,000,000회를 세어 1클록 길이의 1초 tick을 만든다. 카운트의 앞 절반에는 blink=1, 뒤 절반에는 blink=0이므로 점멸 주기는 1초다.

• traffic_sequence는 현재 방향 direction과 단계 phase를 저장한다. 각 방향은 0~12의 13단계를 거친 뒤 다음 방향으로 넘어간다.

• traffic_lights는 방향과 단계에 따라 차량 및 보행자 LED 패턴을 만든다.

• traffic은 세 하위 모듈을 연결하며 led_left는 led_green과 같은 패턴을 출력한다.

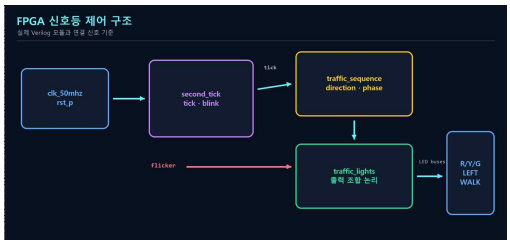


그림 10. 신호등 회로의 계층 구조와 상태 순서

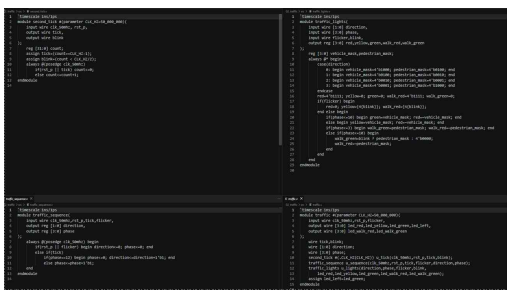


그림 11(a). 사용자가 직접 캡처한 신호등 Verilog 설계 코드

```
`timescale 1ns/1ps
module tb;
    reg clk=0,rst=1,flicker=0;
    always #5 clk=~clk;
    wire [3:0] red,yellow,green,left,wr,wg;
    traffic #(.CLK_HZ(4)) dut(clk,rst,flicker,red,yellow,green,left,wr,wg);
    integer direction,phase,subcycle;
    reg [3:0] vm,pm,er,ey,eg,ewr,ewg;
    task step; begin @(posedge clk); #1; end endtask
    task check;
        begin
            case(direction)
                0: begin vm=8; pm=4; end
                1: begin vm=4; pm=2; end
                2: begin vm=2; pm=1; end
                3: begin vm=1; pm=8; end
            endcase
            er=~vm; ey=0; eg=0; ewr=15; ewg=0;
            if(phase<11) eg=vm; else ey=vm;
            if(phase<4) begin ewg=pm; ewr=~pm; end
            else if(phase<11) begin ewr=~pm; if(subcycle<2) ewg=pm; end
            if({red,yellow,green,wr,wg} !== {er,ey,eg,ewr,ewg})
                $fatal(1,"traffic dir=%0d phase=%0d sub=%0d",direction,phase,subcycle);
            if(left !== green) $fatal(1,"left indicator");
        end
    endtask
    initial begin
        $dumpfile("wave.vcd");
        $dumpvars(0,tb);
        step; @(negedge clk); rst=0;
        for(direction=0;direction<4;direction=direction+1)
            for(phase=0;phase<13;phase=phase+1)
                for(subcycle=0;subcycle<4;subcycle=subcycle+1) begin
                    check; step;
                end
        direction=0; phase=0; subcycle=0; check;
        @(negedge clk); flicker=1; step;
        repeat(8) begin
            if(red !== 0 || green !== 0 || wg !== 0) $fatal(1,"flicker outputs");
            if(yellow !== wr || !(yellow===0 || yellow===15)) $fatal(1,"flicker blink");
            step;
        end
        @(negedge clk); flicker=0; rst=1; step;
        direction=0; phase=0; subcycle=0; check;
        $display("PASS traffic: full cycle, pedestrian blink, flicker, reset");
        $finish;
    end
    initial begin #100000; $fatal(1,"timeout"); end
endmodule
```

그림 11(b). 사용자가 직접 캡처한 신호등 self-checking 테스트벤치

5.2 방향 선택값

direction	차량 진행 마스크	보행자 진행 마스크
0	4'b1000	4'b0100
1	4'b0100	4'b0010
2	4'b0010	4'b0001
3	4'b0001	4'b1000

표 5. 신호등 방향 선택 마스크

현재 진행 방향을 제외한 차량 신호는 적색이고, 현재 보행 방향을 제외한 보행 신호도 적색이다. 출력값 1은 논리적인 켜짐을 뜻하며 실제 장비 LED가 LOW

활성인 경우에는 핀 연결 단계에서 극성을 맞춘다.

5.3 단계별 예상 동작

phase	지속 구간	진행 방향 차량	진행 보행자	나머지 신호
0~3	4초	녹색	녹색	적색
4~10	7초	녹색	0.5초 간격 점멸 녹색	적색
11~12	2초	황색	적색	적색

표 6. 신호등 단계별 예상 동작

한 방향은 13초이고 네 방향 전체 순환은 52초다. 단계 12가 끝나면 phase는 0으로 돌아가고 direction은 1 증가한다. 방향 3 다음에는 2비트 오버플로에 의해 방향 0으로 돌아간다.

flicker=1인 비상 점멸 모드에서는 direction=0, phase=0으로 초기화하며 차량 적색·녹색 및 보행 녹색은 모두 꺼진다. 네 방향 황색과 네 방향 보행 적색은 blink에 따라 동시에 점멸한다. flicker를 해제하면 방향 0, 단계 0부터 정상 순환을 다시 시작한다.

5.4 테스트벤치 검증

테스트벤치에서는 CLK_HZ=4로 축소한다. 1초에 해당하는 네 클록을 두 클록 HIGH, 두 클록 LOW의 점멸 구간으로 나누어 다음 항목을 검사한다.

1. 네 방향 각각의 13단계 전체 출력
2. phase 4~10에서 보행 녹색 점멸
3. led_left == led_green
4. 비상 점멸 모드에서 황색과 보행 적색의 동시 점멸
5. 리셋 후 방향 0, 단계 0 출력 복귀

예상 종료 메시지는 다음과 같다.

```
PASS traffic: full cycle, pedestrian blink, flicker, reset
```



그림 12. 사용자가 직접 실행한 신호등 self-checking 시뮬레이션 PASS 결과



그림 13. 사용자가 직접 확인한 신호등 direction·phase·차량 신호 전환 파형

6. FPGA · 외부 ADC 제어

6.1 회로 구조와 검증 범위

이 실습은 STM32 내부 ADC를 사용하는 실습이 아니다. FPGA가 외부 변환 장치에 16비트 명령을 보내고 16비트 직렬 응답을 받아 디지털 결과를 표시한다.

- adc_command는 3비트 채널 선택값으로 16비트 명령을 조합한다.
- serial_frame16은 HIGH 대기 직렬 클록을 만들고 16비트를 송수신한다.
- adc는 프레임을 반복 시작하고, 완료 시 수신 워드의 하위 12비트를 sample에 저장하며 한 클록 길이의 data_valid를 발생시킨다.
- led_out에는 sample[11:4]를 표시한다.

시뮬레이션의 ADC 모델은 디지털 직렬 응답만 제공한다. 실제 입력 전압, 변환 정확도, 초기화 시간, 채널을 바꾼 뒤 유효 결과가 나오기까지의 지연은 실제 장치와 데이터시트로 확인해야 한다.

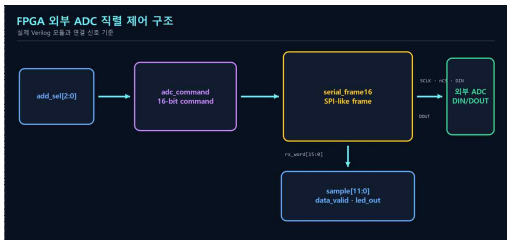


그림 14. ADC 명령·16비트 직렬 송수신·결과 저장 구조



그림 15(a). 사용자가 직접 캡처한 외부 ADC 제어 Verilog 설계 코드

```
`timescale 1ns/1ps
module tb;
    reg clk=0,rst=1;
    always #5 clk=~clk;
    reg [2:0] channel=2;
    reg miso=0;
    wire sclk,cs_n,mosi,valid;
    wire [11:0] sample;
    wire [7:0] led;
    adc #(.HALF_CYCLES(2)) dut(clk,rst,channel,sclk,cs_n,mosi,miso,sample,valid,led);
    reg [15:0] received_command;
    reg [15:0] reply=16'hA5C3;
    reg [2:0] frame_channel;
    integer bit_no,received_bits,frames=0;
    always @(negedge cs_n) begin
        bit_no=15; received_bits=0; received_command=0; frame_channel=channel;
    end
    always @(negedge sclk)
        if(!cs_n) miso=reply[bit_no];
    always @(posedge sclk)
        if(!rst && received_bits<16) begin
            received_command={received_command[14:0],mosi};
            received_bits=received_bits+1;
            bit_no=bit_no-1;
        end
    always @(posedge valid) begin
        #1;
        if(received_bits != 16) $fatal(1,"ADC frame length");
        if(received_command !== (16'hA370 | (frame_channel << 10)))
            $fatal(1,"ADC command channel=%0d cmd=%h",frame_channel,received_command);
        if(sample !== reply[11:0] || led !== reply[11:4]) $fatal(1,"ADC sample");
        frames=frames+1;
    end
    initial begin
        $dumpfile("wave.vcd");
        $dumpvars(0,tb);
        repeat(2) @(negedge clk);
        if(cs_n!==1 || sclk!==1 || sample!==0) $fatal(1,"ADC reset");
        rst=0;
        wait(frames==1); @(negedge clk); channel=5;
        wait(frames==2); @(negedge clk); channel=7;
        wait(frames==3); @(negedge clk); rst=1;
        @(posedge clk); #1;
        if(cs_n!==1 || sclk!==1 || sample!==0 || valid!==0) $fatal(1,"ADC reset after data");
        $display("PASS ADC: 3 serial frames, command bits, sample, reset");
        $finish;
    end
    initial begin #100000; $fatal(1,"timeout"); end
endmodule
```

그림 15(b). 사용자가 직접 캡처한 ADC self-checking 테스트벤치

6.2 채널별 명령 예상값

명령 형식은 {3'b101, channel, 2'b11, 1'b0, 3'b111, 4'b0000}이며, 식으로 쓰면 16'hA370 | (channel << 10)이다.

채널 add_sel	전송 명령
0	16'hA370
1	16'hA770
2	16'hAB70
3	16'hAF70
4	16'hB370
5	16'hB770
6	16'hBB70

채널 add_sel	전송 명령
7	16'hBF70

표 7. ADC 채널별 16비트 전송 명령

6.3 직렬 프레임 동작

리셋 상태에서 adc_ncs=1, adc_sclk=1, sample=0, data_valid=0이다. 전송을 시작하면 adc_ncs가 0이 되고 최상위 비트부터 adc_din으로 보낸다. adc_sclk의 하강 에지 구간에서 다음 출력 비트를 준비하고 상승 에지마다 adc_dout을 한 비트씩 수신한다. 16번째 비트를 받은 뒤 adc_ncs를 1로 올리고, done과 data_valid를 한 클록 동안 1로 만든다. 기본 HALF_CYCLES=25에서는 50 MHz 클록 25주기마다 직렬 클록이 반전되므로 직렬 클록의 한 주기는 50 FPGA 클록, 즉 약 1 MHz다. 테스트벤치에서는 빠른 검증을 위해 HALF_CYCLES=2를 사용한다.

6.4 테스트벤치 검증

ADC 모델의 응답 워드를 16'hA5C3으로 고정한다. 따라서 저장할 결과는 sample=12'h5C3, LED 출력은 8'h5C가 되어야 한다. 채널을 2 → 5 → 7로 변경하며 다음 세 프레임을 검사한다.

프레임	채널	예상 전송 명령	예상 수신 결과	예상 LED
1	2	16'hAB70	12'h5C3	8'h5C
2	5	16'hB770	12'h5C3	8'h5C
3	7	16'hBF70	12'h5C3	8'h5C

표 8. ADC 사전 시뮬레이션의 프레임별 예상값

각 프레임이 정확히 16비트인지, 명령에 현재 채널이 반영되는지, 완료 후 결과와 LED가 맞는지, 실행 중 다시 리셋했을 때 초기값으로 돌아가는지를 검사한다. 예상 종료 메시지는 다음과 같다.

PASS ADC: 3 serial frames, command bits, sample, reset



그림 16. 사용자가 직접 실행한 ADC self-checking 시뮬레이션 PASS 결과

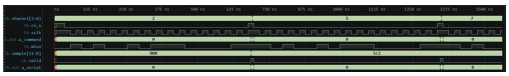


그림 17. 사용자가 직접 확인한 ADC channel·CS·SCLK·MISO·sample·valid 파형

본 실험에서는 먼저 디지털 파형을 다시 확인하고, 그 뒤 교재와 장비의 핀 설명에 맞춰 외부 ADC를 연결한다. 입력 전압을 바꾸며 sample과 led_out이 변하는지 확인한다. 핀맵 검증 전에는 XDC를 추가하지 않는다.

7. 본 실험 전·후 확인 항목

7.1 본 실험 전 완료할 항목

항목	완료 기준
STM32 과제3	ISR 기반 코드 작성, 빌드 성공, 폴링 입력·시간 처리 제거
STM32 과제4	PWM 시작 및 250·500·750 순환 코드 작성, 빌드 성공
피아노	설계 모듈과 TB 작성, 자체 검사 시뮬레이션 PASS
신호등	설계 모듈과 TB 작성, 전체 순서·점멸·리셋 PASS
ADC	설계 모듈과 TB 작성, 3개 프레임·명령·수신·리셋 PASS
저장소	소스, TB, 실행 방법, 보고서 PDF를 경로별로 정리
미완료로 표시할 항목	XDC, Vivado 합성·구현·타이밍·비트스트림, 실제 보드 결과

표 9. 본 실험 전 완료 기준

7.2 본 실험에서 수행할 순서

1. STM32 과제3 프로그램을 빌드하고 보드에 기록한다.
2. 버튼 인터럽트와 TIM6 인터럽트가 실행되는지 디버거로 확인한다.
3. 버튼 입력별 PA5 상태 변경 간격을 측정하고 영상을 촬영한다.
4. STM32 과제4 코드로 전환해 빌드하고 보드에 기록한다.
5. 디버거로 PWM 비교값을 확인한 뒤 중단점을 해제한다.
6. PB4 파형의 1 kHz 주파수와 25%·50%·75% HIGH 시간을 측정한다.
7. FPGA 세 회로를 각각 별도 Vivado 프로젝트로 만들고 설계 소스와 TB를 추가한다.
8. Behavioral Simulation에서 PASS 메시지와 파형을 확보한다.
9. 교재 및 장비의 핀맵을 확인한 뒤 각 프로젝트에 XDC를 추가한다.
10. Synthesis → Implementation → DRC → Timing → Bitstream을 순서대로 확인한다.
11. FPGA 보드에 기록하고 피아노, 신호등, ADC를 각각 시연한다.

7.3 결과보고서에 추가할 증거

• STM32 두 과제의 빌드 성공 화면
• 중단점, 한 줄 실행, Variables/Watch 화면
• PA5 간격 순환 영상과 PB4의 25%·50%·75% 계측 파형
• FPGA 세 프로젝트의 Sources, Project Summary, 시뮬레이션 파형과 PASS 메시지
• 검증된 XDC의 핀 제약 화면 또는 파일 링크
• Synthesis/Implementation/DRC/Timing/Bitstream 성공 화면
• 피아노 8음, 신호등 정상·점멸 순서, ADC 입력 변화 시연 영상
• 예상값과 실제 측정값의 비교표, 오차와 문제 해결 내용

8. GitHub 및 재현 정보

• 저장소 URL: https://github.com/dalkomys777/fpga_lab5

• 기준 브랜치: main

• 예비보고서 PDF: https://github.com/dalkomys777/fpga_lab5/blob/main/reports/pre/%EC%98%88%EB%B9%84%EB%A6%AC%ED%8F%AC%ED%8A%B8_LAB5%202025440031%EA%B9%80%EC%98%81%EC%84%9C.pdf

• 증빙 기준 태그: prelab-v1.1.0

• 증빙 기준 커밋: 073734b9992a04b6ea751aab4345556ab98dc271

• 증빙 커밋 링크: https://github.com/dalkomys777/fpga_lab5/commit/073734b9992a04b6ea751aab4345556ab98dc271

• 증거사진 폴더: https://github.com/dalkomys777/fpga_lab5/tree/073734b9992a04b6ea751aab4345556ab98dc271/evidence/report_captures

8.1 GitHub 증거사진 링크

아래 링크는 사용자 직접 캡처 17장을 증빙 기준 커밋에 고정한 주소이다. main 브랜치가 변경되어도 같은 이미지를 확인할 수 있다.

• 공통 작업공간: https://github.com/dalkomys777/fpga_lab5/blob/073734b9992a04b6ea751aab4345556ab98dc271/evidence/report_captures/workspace.png

• STM32 과제3 코드 전반부: https://github.com/dalkomys777/fpga_lab5/blob/073734b9992a04b6ea751aab4345556ab98dc271/evidence/report_captures/stm32_task3_code_top.png

• STM32 과제3 코드 후반부: https://github.com/dalkomys777/fpga_lab5/blob/073734b9992a04b6ea751aab4345556ab98dc271/evidence/report_captures/stm32_task3_code_bottom.png

• STM32 과제4 코드 전반부: https://github.com/dalkomys777/fpga_lab5/blob/073734b9992a04b6ea751aab4345556ab98dc271/evidence/report_captures/stm32_task4_code_top.png

• STM32 과제4 코드 후반부: https://github.com/dalkomys777/fpga_lab5/blob/073734b9992a04b6ea751aab4345556ab98dc271/evidence/report_captures/stm32_task4_code_bottom.png

• 피아노 설계 코드: https://github.com/dalkomys777/fpga_lab5/blob/073734b9992a04b6ea751aab4345556ab98dc271/evidence/report_captures/piano_design.png

• 피아노 테스트벤치:

https://github.com/dalkomys777/fpga_lab5/blob/073734b9992a04b6ea751aab4345556ab98dc271/evidence/report_captures/piano_testbench.png

• 피아노 PASS 결과: https://github.com/dalkomys777/fpga_lab5/blob/073734b9992a04b6ea751aab4345556ab98dc271/evidence/report_captures/piano_pass.png

• 피아노 파형: https://github.com/dalkomys777/fpga_lab5/blob/073734b9992a04b6ea751aab4345556ab98dc271/evidence/report_captures/piano_wave.png

• 교통신호 설계 코드: https://github.com/dalkomys777/fpga_lab5/blob/073734b9992a04b6ea751aab4345556ab98dc271/evidence/report_captures/traffic_design.png

• 교통신호 테스트벤치: https://github.com/dalkomys777/fpga_lab5/blob/073734b9992a04b6ea751aab4345556ab98dc271/evidence/report_captures/traffic_testbench.png

• 교통신호 PASS 결과: https://github.com/dalkomys777/fpga_lab5/blob/073734b9992a04b6ea751aab4345556ab98dc271/evidence/report_captures/traffic_pass.png

• 교통신호 파형: https://github.com/dalkomys777/fpga_lab5/blob/073734b9992a04b6ea751aab4345556ab98dc271/evidence/report_captures/traffic_wave.png

• ADC 설계 코드: https://github.com/dalkomys777/fpga_lab5/blob/073734b9992a04b6ea751aab4345556ab98dc271/evidence/report_captures/adc_design.png

• ADC 테스트벤치: https://github.com/dalkomys777/fpga_lab5/blob/073734b9992a04b6ea751aab4345556ab98dc271/evidence/report_captures/adc_testbench.png

• ADC PASS 결과: https://github.com/dalkomys777/fpga_lab5/blob/073734b9992a04b6ea751aab4345556ab98dc271/evidence/report_captures/adc_pass.png

• ADC 파형: https://github.com/dalkomys777/fpga_lab5/blob/073734b9992a04b6ea751aab4345556ab98dc271/evidence/report_captures/adc_wave.png

8.2 파일 구조

실습	설계 또는 프로그램	테스트벤치·검증
STM32 과제3	stm32/task3_interrupt_led/ece2_474_routine.c	빌드 로그, 본 실험 영상
STM32 과제4	stm32/task4_pwm_duty/ece2_474_routine.c	빌드 로그, 디버거·계측 사진
피아노	circuits/01_piano/src/*.v	circuits/01_piano/sim/tb_design.v
신호등	circuits/02_traffic/src/*.v	circuits/02_traffic/sim/tb_design.v
ADC	circuits/03_adc/src/*.v	circuits/03_adc/sim/tb_design.v
보고서	reports/pre/	PDF와 필요한 그림
본 실험 증거	evidence/	회로별 사진·영상·로그

표 10. LAB5 저장소 파일 구조와 검증 위치

8.3 재현 조건

• 재현 방법: git checkout prelab-v1.1.0 후 LAB5.code-workspace를 열어 회로별 시뮬레이션 작업을 실행한다.
• FPGA 설계 시뮬레이션 최상위: tb
• 실제 FPGA 설계 최상위: 피아노 piano, 신호등 traffic, ADC adc
• 시뮬레이션에서는 각 회로를 한 번에 하나씩 실행한다.
• 테스트벤치는 보드에 합성하지 않는다.
• 클록 축소 매개변수는 시뮬레이션 전용이며 실제 보드의 50 MHz 설정과 구분한다.
• XDC는 교재와 장비의 핀맵을 확인한 후 별도 커밋으로 추가한다.
• 결과보고서에는 실제 사용한 태그와 커밋 SHA를 고정해 같은 소스를 다시 확인할 수 있게 한다.

9. 참고문헌

[1] Joseph Yiu, *The Definitive Guide to ARM Cortex-M3 and Cortex-M4 Processors*, 3rd ed., Newnes, 2013, ISBN 978-0-12-408082-9.

[2] Pong P. Chu, *FPGA Prototyping by Verilog Examples: Xilinx Spartan-3 Version*, Wiley, 2008, ISBN 978-0-470-18532-2.

[3] STMicroelectronics, *RM0440: STM32G4 Series Advanced Arm-Based 32-bit MCUs Reference Manual*, 인터럽트, TIM6 및 TIM3 관련 절.

[4] ECE2-474 실습 자료, *STM32_4week_plan.pdf*, 10월 19일 STM32_1 및 FPGA 실습 일정.

[5] ECE2-474 실습 문서, *STM32_1 · 인터럽트와 PWM* 및 *FPGA 추가 실습 · 모듈과 테스트벤치*.